

TITLE: Network Automation with Python: Proposing a Closed-Loop System with SNMP Data

AUTHOR: Ana Luiza Miranda

ABSTRACT:

In this work, we explore the use of Python and the Simple Network Management Protocol (SNMP) for network automation and cybersecurity. We discuss the concepts, use cases, improvements, challenges, and related technologies of network automation. To illustrate a use case, we propose a closed-loop network automation system that integrates data collection, processing, analysis, and troubleshooting. Utilizing Python libraries such as PySNMP, pandas, scikit-learn, and Flask, we develop a machine learning-based anomaly detection model to monitor and enhance network security and performance using SNMP data related to the Interface and protocols: IP, ICMP, TCP, and UDP. The results demonstrate the model's effectiveness in detecting various types of attacks, highlighting Python's versatility and power for network automation. Additionally, we discuss the usefulness of SNMP data for monitoring and diagnosing networks and suggest future improvements, including real-world implementation and fine-tuning of the machine learning model.

KEYWORDS: Network Automation; Python; SNMP; Anomaly Detection; Machine Learning; Closed-Loop System.

INTRODUCTION

In the actual context of the rising use of cloud services, IoT services ("Internet of Things"), and big data, traditional network architectures need to keep pace with these changes. In this situation, the concept of network automation seem to be a path to solve this problem. Accordingly to the Open Networking Foundation:

Many conventional networks are hierarchical, built with tiers of Ethernet switches arranged in a tree structure. This design made sense when client-server computing was dominant, but such a static architecture is ill-suited to the dynamic computing and storage needs of today's enterprise data centers, campuses, and carrier environments [1].

Network automation not only refers to configuring devices but to the use of software and tools to perform network management and operation tasks in an

automated manner, without manual intervention. “Network programmability allows for reliable and dynamic infrastructure configurations by automating deployments, simplifying the network, and reducing human-generated errors” [2]. It also can be used to configuring and provisioning network devices, monitoring and analyzing performance, detecting and resolving failures, implementing security policies, among other activities.

Known by its flexibility and high-level language, Python is a popular choice when developing network automation activities. “Python is an essential programming language that has gained significant popularity in the networking industry due to its simplicity, versatility, and extensive library of modules” [3]. This way, Python can be considered a great first choice for network engineers learn to code for three main reasons: the first one is the simplicity when dealing with variables and functions, it doesn’t require to be defined before using them. The second one is that Python is easy to read, and the last one is “that network vendors and open source projects are building a great set of libraries and tools using Python” [4].

The general objective of this paper is to discuss network automation using Python, and to illustrate a use case, we proposed a closed-loop system, consisting of data collection (from the SNMP protocol - Simple Network Management Protocol), data processing, and troubleshooting (using advanced analytics). The specific objectives are:

- Present network automation, its concept, types, use cases, and related technologies;
- Propose an automated network diagnostic cycle (data collection, processing, and troubleshooting) using the SNMP protocol;
- Discuss the use of the Python programming language and its tools (libraries and frameworks) such as PySNMP in this cycle.

Our methodological approach to develop this research will be a bibliographic review made from scientific texts and relevant books on the topic. We will also use the documentation from the dataset and the Python tools (libraries and frameworks).

Additionally, we will incorporate examples of Python scripts and data analysis, visualization, and modeling into our discussion. All of this is aimed at illustrating and demonstrating the closed-loop proposed in our main objective.¹

LITERATURE REVIEW

As mentioned previously, network automation is commonly associated with optimizing time for equipment configuration; however, this concept goes beyond that. Through scripts, software, and specialized tools, we can automate the management, configuration, provisioning, and monitoring of network devices. All of this is done efficiently and precisely, reducing the risk of errors.

According to Cisco's blog, "Network automation is the process of automating the configuring, managing, testing, deploying, and operating of physical and virtual devices within a network" [5]. This way, the application of network automation can lead to several improvements in the network administration, such as simplified architectures, deterministic outcomes (using automation, the network has a more predictable behavior, avoiding configuration errors and inconsistencies in the network), and enhanced security and risk reduction ("Automation enables sharing of responsibility across the team and makes the process more effective and less prone to introducing problems because it enforces extra human review") [4].

Nonetheless, the use of network automation can also bring disadvantages, if not well-established and planned, it can increase the complexity of the management and maintenance. As George Millios highlighted:

To automate a network effectively the network engineers must have deep understanding of how network devices work and behave to implement automation to network devices. [...] Network automation minimizes the human error factor but does not eradicate them. When automating an error, the impact is much bigger than a human making a configuration error in one device [6].

Besides that, when implementing network automation, other difficulties may arise, such as the compatibility of devices to these automated tasks, and potentially the need to purchase new ones. Additionally, the absence of a

¹ The proposed scripts, as well as the code and results of the data analysis and construction of the classification model can be consulted at the link: <https://github.com/ana0212/Closed-Loop-System-with-SNMP-Data>

universally accepted, vendor-neutral schema, coupled with a cost-effective testing environment, can pose challenges [6].

Despite these challenges, network automation has multiple use cases, such as:

- **Device provisioning:** We can create device configuration files predictably and reliably, for initial device provisioning and then send them to network devices;
- **Data Collection and Enrichment:** We can obtain data in a personalized way, exactly as desired, and at the desired time, format it and also analyze it;
- **Migrations:** During a migration, we can identify tasks that can be automated to simplify and speed up the process. Network automation allows you to automate specific migration tasks, adapting them to the unique needs of your environment;
- **Configuration Management:** This is the most common type of automation. “We define configuration management as deploying, pushing, and managing the configuration state of a device”. This ranges from basic interface definitions to more elaborate procedures such as configuring ToR switches, firewalls, load balancers, and advanced security infrastructure to support the deployment of three-tier applications;
- **Configuration Compliance:** This is a low-risk use case. Network automation plays a crucial role in reducing risks associated with configuration changes, allowing you to start with data collection, monitoring, and initial configuration, thus mitigating potential problems. Using collected data, compliance checks, and configuration validation represent low-risk use cases, providing incremental control over deployed tools and enabling event-driven automation for complex operations, with the potential for automatic remediation;
- **State Validation:** The next step after configuration compliance, which adds more control of the automation process;
- **Reporting:** With the data collected we can create reports, adapting to our necessities;
- **Troubleshooting:** We can use advanced analytics to break/fix problems [4].

Some of these use cases can be combined, making a closed-loop system, comprising data collection, data processing, data analysis, and troubleshooting

[4]. With this structure, we are going to develop the practical part of this article (Figure 1).

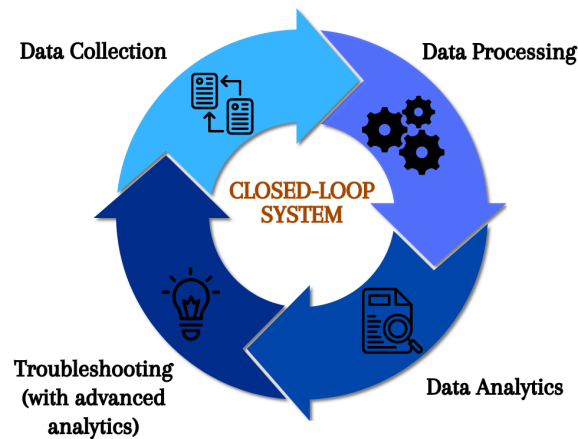


Figure 1. Steps of the closed-loop system

- **Data Collection:** In this step, data is automatically collected from the network using the SNMP protocol to obtain information from network devices. To exemplify, we will use PySNMP scripts (a Python library that provides an interface for interacting with network devices supporting the SNMP protocol) to collect data from a fictitious network;
- **Data Processing:** The collected data is processed and organized for analysis. This may involve data cleaning, and transformation to ensure it is in a suitable format for subsequent analysis. In this part, we'll utilize one of Python's most popular data manipulation libraries, pandas;
- **Data Analysis:** At this stage, we'll analyze the SNMP MIBS 2016 dataset [7] to identify patterns, trends, or anomalies. In addition to continuing to use pandas, we will employ the Matplotlib, Seaborn, and Scikit-learn libraries to visualize the data. We will also use Jinja2, in our proposed script to automatically generate a report;
- **Troubleshooting (with advanced analytics):** This step involves applying advanced data analysis techniques with this dataset to diagnose problems or anomalies in the network. At this stage, we'll use machine learning techniques, like classification, to identify anomalies. The machine learning algorithm that we chose was SVM (Support Vector Machine), which we are going to discuss later on this work. We are also going to propose a script to

report an alert with some suggestions for actions based on the model's prediction.

Closed-loop automation systems are an example of how Artificial Intelligence (AI) methods can benefit network management through Machine learning algorithms:

Closed-loop automation systems help transform network and IT operations by using AI-driven automation to detect anomalies, determine resolution, and implement the required changes in a highly automated framework [...] Closed-loop automation helps solve many problems before they even become issues [8].

These systems can have capabilities such as intelligent alerts (in case of failures), predictive planning (behavior prediction of the network), root-cause analysis (to find the most likely cause for an incident, ensure accuracy, and implement the required remediation steps), and the one we are going to discuss more in this paper, anomaly detection. The use cases are also diverse: traffic-flow optimization, security, and container orchestration with Kubernetes, for example [8].

The benefits of this implementation can also be diverse, such as improved network reliability ("With the ability to automatically identify issues on our network, not only can we automatically fix the issue but we can also apply alternate network paths that can help mediate the effects while the fix is applied"), better customer experience, increased productivity, and better network performance (decreasing mean-time to resolution for incidents) [8].

Another crucial subject in this paper is the SNMP protocol, from which we are going to collect all the network data necessary for our closed-loop system. We chose this protocol to be our data source because it "is a widely used Layer7 protocol that has been around for decades, it is responsible for exchanging management information between network devices"[6]. It also is vendor-neutral and free of licensure and copyright constraints, being standardized and implemented in almost all network devices.

This protocol has four main components: "SNMP Manager, SNMP Agent, Managed Devices, and Management Information Base (MIB)"[6]. Since we are using SNMP to collect data, our focus is on the MIB, where:

Management information is represented as a collection of managed objects that together form a virtual information store [...] A MIB object might be a counter, descriptive information such as software version; status information such as whether

or not a device is healthy, or protocol-specific information such as a routing path to a destination. MIB objects thus define the management information maintained by a managed node. Related MIB objects are gathered into so-called MIB modules[6].

In this section, we discussed, based on the literature in the area, the main points of this paper: network automation, closed-loop system for managing networks in an automated way using AI methods, and the SNMP protocol. In the next part of this work, we will use these concepts to propose a closed-loop system to detect anomalies in the network based on SNMP data.

RESULTS

1. Data Collection:

As one of the main network management protocols, SNMP can provide a wealth of useful data to monitor a network. With the MIB of a device, we can access data related to various other important protocols (like TCP, UDP, IP, and ICMP), giving us a view of many layers of the OSI model of the network:

Each network device being managed exposes a set of data that can be collected and configured via the SNMP agent. This set of data that is managed through SNMP is described and modeled through MIBs. Only if an MIB is exposing a certain feature can it be monitored or managed. This includes making configuration changes through SNMP. Often overlooked, SNMP not only supports GetRequests for monitoring but also supports SetRequests for manipulating objects and variables exposed through MIBs [4].

As mentioned before, SNMP is a well-established protocol that has been used for decades, but it wasn't designed for real-time management [6] and network automation. Therefore, we need other tools to use this data for these purposes, and Python offers plenty of libraries and frameworks, such as Ansible, NAPALM, and Nornir. In our context, we are going to use the PySNMP library.

PySNMP is "a pure-Python, open source and free implementation of v1/v2c/v3 SNMP engine distributed under 2-clause BSD license" [9] and provides an interface to interact with network devices that support the SNMP protocol. Its latest release is version 4.4.12, and includes (among others) these features [9]:

- Includes an SMI framework that resolves MIB information and implements SMI-managed objects, facilitating detailed interpretation and utilization of network management data;

- Provides a complete SNMP entity implementation, ensuring that all essential SNMP functionality and operations are available for comprehensive interaction with network devices;
- It has an extensible network transport framework, supporting UDP/IPv4 and UDP/IPv6, which ensures efficient communication with network devices in various configurations;
- Has an I/O API based on asynchronous sockets allows data collection asynchronously, increasing efficiency and responsiveness in real-time data collection;
- Integration with PySMI for dynamic MIB compilation automates the translation and use of MIBs, making it easier to interpret and manipulate data from network devices;
- Has built-in instrumentation that exposes protocol engine operations, providing internal tools that offer visibility and control over SNMP protocol operations, which is useful for debugging and optimization[9].

PySNMP has a 'SNMP Engine' that "is a central object that controls the other components of the SNMP system"[9]. Additionally, it has five other internal components:

- **Transport Subsystem:** Used to send and receive SNMP messages from the network;
- **Message Dispatcher and PDU:** This component handles SNMP message processing activities. Its main responsibilities include sending SNMP Application PDUs through various subsystems to the Transport Dispatcher and passing SNMP messages from the network to SNMP Applications;
- **Message Processing Modules:** These modules analyze and construct messages, invoking security services whenever necessary;
- **Message Security Modules:** Responsible for message authentication and/or encryption;
- **Access Control Subsystem:** Utilizes Local Configuration Datastore (LCD) information to authorize remote access to monitored objects, especially when running in the agent role. The LCD comprises a collection of modules and MIB objects that the SNMP engine uses to maintain its configuration and operational statistics.

In our proposed data collection script, we used PySNMP with a fictitious network, collecting data from the MIB Interface of a router (through the OID). This script is based on the dataset that we used for our data analysis and troubleshooting sections [7]. In this example, we demonstrated the various components of the PySNMP architecture in action. Also, we scheduled to collect data every 15 seconds, following the dataset manners [11], and saved the .csv files with a datestamp, so we can use it in the other closed-loop system steps.

2. Data Processing:

From this data collection, we can store the data in native Python structures, such as lists and dictionaries, and then clean and standardize the data to meet our needs and create tables for analysis. In this context, we can use one of the most popular libraries for data manipulation in Python, pandas:

pandas is a Python package providing fast, flexible, and expressive data structures designed to make working with “relational” or “labeled” data both easy and intuitive. It aims to be the fundamental high-level building block for doing practical, real-world data analysis in Python. Additionally, it has the broader goal of becoming the most powerful and flexible open-source data analysis/manipulation tool available in any language. It is already well on its way toward this goal [12].

Pandas can handle different kinds of data, such as data from different sources (e.g., SQL tables and Excel spreadsheets), time series data, arbitrary matrix data, etc. It also has two primary data structures: Series (1-dimensional) and DataFrame (2-dimensional), (pandas documentation). In our case, we are going to use the DataFrame [12].

In the context of processing the collected SNMP data, we can remove null values, check for invalid data (like negative values), normalize and standardize the values, and correct the data types. For all these steps, pandas provide functions and methods that facilitate our work. We also created an example of a script for this step, to be incorporated into the data collection script. It has some basic data cleaning and validation procedures, like checking duplicates and data types.

3. Data Analysis

At this step and the next (troubleshooting with advanced analytics), to better exemplify our closed-loop cycle, we will use an SNMP dataset. In their work,

Alkasassbeh and Mouhammd collected data from a fictitious network across five MIB groups: Interface, TCP, UDP, IP, and ICMP [7]. The data was collected from the router of this network, which had the following topology: one router, two switches, and two subnets. The first subnet contained five PCs, one of which acted as the attacker. The second subnet also contained five PCs, one of which acted as the server (victim). Besides normal traffic, the network faced six different DoS (Denial of Service) attacks (TCP-SYN, UDP flood, ICMP-echo, HTTP flood, Slowloris, and Slowpost) and one brute-force attack. The data is labeled according to the different network states [11].

We chose this dataset because it not only collects data from our primary source, the SNMP protocol, but the groups of MIB variables span a wide range of the OSI model, from the link layer to the transport layer. This provides an extensive view of network performance. The multi-layer focus indicates the intent to monitor both data traffic (IP, TCP, UDP) and network control and management (ICMP) events, in addition to overall device packet counts in the interface group.

The dataset dictionary describes 34 MIB (Management Information Base) variables collected from a router device, categorized into five MIB-II groups: Interface, IP, TCP, UDP, and ICMP. Each variable represents a specific metric related to the performance and state of the network, and all are of type counter 32, a counter that continuously increments up to a limit before restarting [11]. The dataset contains 4,998 entries, but we are going to use only 3,538, which correspond to the normal functioning of the network and the first four DoS attacks: TCP-SYN, UDP flood, ICMP-echo, and HTTP flood.

The first step of our analysis was to split the dataset into training (70%) and testing (30%). We used the scikit-learn tool *train_test_split*. Additionally, since the classes were imbalanced (960 entries for 'tcp-syn', 773 for 'udp-flood', 632 for 'icmp-echo', 600 for 'normal', and 573 for 'httpFlood'), we used the parameter "stratify" to ensure each class is represented in our analysis and future modeling. Scikit-learn is a widely used open-source Python library with tools for "predictive data analysis [...] built on NumPy, SciPy, and matplotlib" [13]. We are going to use many of its tools to process, transform, and model data.

In our univariate analysis, we separated the training dataset by classes and described the principal descriptive statistics of each one based on the protocol columns associated with each attack (TCP with 'tcp-syn', UDP with 'udp-flood', IP

with 'httpFlood', and ICMP with 'icmp-echo'). We then compared these results with the 'normal' class for these columns. Based on the median and standard deviation values, we noticed that most columns changed their behaviors during the attacks. Here are some insights from this analysis:

1. TCP Analysis:

- Comparing the TCP-related columns during 'normal' and 'tcp-syn' classes, the only column that maintains the median and standard deviation in the same range is 'tcpCurrEstab'. This possibly indicates that the number of connections that are established doesn't change during the 'tcp-syn' attack;
- Changes in all other columns indicate that a 'tcp-syn' attack can bring instability to the metrics of network traffic, resulting in erratic and inconsistent behavior in the TCP statistics.

2. UDP Analysis:

- Comparing the 'normal' and 'udp-flood' statistics in the UDP columns, we noticed that the mean of errors in the UDP datagrams received ('udpInErrors' column) is smaller during an attack ('normal': 1.53 and 'udp-flood': 0.82). This may indicate that during an attack, most of the datagrams are valid or that the system is not able to register all of them due to overload;
- Another interesting point is that the mean of the UDP datagrams received on unused ports ('udpNoPorts' column) is slightly smaller during the attack, but the difference is not significant. This possibly indicates that the attack is more focused on valid ports or follows an attack pattern that doesn't reach many unused ports;
- There is a significant rise in the volume of UDP packets received and sent, and the variability increases during a UDP flood attack. For example, in the 'udpInDatagrams' column, the standard deviation is 89,395 in the 'normal' class and 112,149 in the 'udp-flood' class.

3. IP Analysis:

- Analyzing the IP group, we noticed a significant rise in the mean in the IP errors address received column ('ipInAddrErrors' column), from 3.16 in the 'normal' class to 51.23 in the 'httpFlood' class. This suggests that most of the packets received have invalid or malformed addresses during the attack, a common characteristic of this kind of DoS. The standard deviation also increases from 3.00 to 51.23;

- Other alterations in the IP group during an HTTP flood attack include a decrease in the number of discarded packets ('ipOutDiscards' and 'ipInDiscards' columns), indicating different prioritization or less capacity to handle processing. The variability of the metrics also indicates that even under overload, the traffic pattern is more uniform in some aspects, such as sent requests and discarded packets.

4. ICMP Analysis:

- In the ICMP group, we noticed greater variability in all the metrics analyzed, suggesting a more erratic and less predictable network traffic behavior, which is common in overload or attack situations. For example, in the 'icmpOutDestUnreaches' column, the standard deviation is 0.99 under normal conditions and 9.1 during the attack;

The second part was the bivariate analysis, where we calculated the correlation between the protocol-related columns and the class of attack it relates to, comparing these correlations with those of the 'normal' class. Here are the highlights:

1. TCP group:

- Looking at the correlation matrix for the TCP group, we found that in normal conditions, the exit resets and retransmitted segments have some negative correlation with other variables. During the TCP-SYN attack, these correlations can invert or decrease, reflecting abnormal behavior. For example, the columns 'tcpOutRsts' and 'tcpPassiveOpens' have correlations of 0.69 in the 'normal' class and -0.98 in the 'tcp-syn' class. Similarly, the columns 'tcpInSegs' and 'tcpEstabResets' have correlations of 0.2 in the 'normal' class and -0.67 in the 'tcp-syn' class.

2. UDP group:

- During a UDP flood attack, the correlation between entry errors ('udpInErrors' column) and other features changes significantly, indicating that the attack impacts the stability of UDP traffic;
- The correlation between 'udpInDatagrams' and 'udpInErrors' is very negative (-0.96) under normal conditions, suggesting an association between increased outgoing UDP datagrams and reduced incoming UDP errors. During a UDP flood attack, this correlation is weaker and positive (0.09),

indicating a much less significant relationship between outgoing datagrams and incoming errors during the attack.

3. IP group:

- In the 'normal' class correlation matrix for the IP group, there are only NaN values for the column 'ipOutNoRoutes' and the other columns. For the 'httpFlood' correlation matrix, there is variation;
- The correlation between received packets and discarded packets changes drastically during the attack. Columns like 'ipInReceives', 'ipInDelivers', 'ipOutRequests', and 'ipOutDiscards' columns have high correlations in the 'normal' class, but these metrics weaken or even change sign during an attack. For example, 'ipInReceives' and 'ipOutDiscards' have a correlation of 0.96 in the 'normal' class and 0.24 in the 'httpFlood' class.

4. ICMP group:

- The correlation between 'icmpOutMsgs' and 'icmpOutDestUnreaches' columns rises considerably during the attack, suggesting that more ICMP messages sent are associated with more destination unreachable messages sent, which wasn't evident in the normal class (the correlation goes from 0.75 in the 'normal' class to 0.91 in the 'icmp-echo' class);
- The correlation between 'icmpInMsgs' and 'icmpOutDestUnreaches' changes sign, going from -0.22 in the 'normal' class to 0.14 in the 'icmp-echo' class. This suggests that the behavior related to the treatment of ICMP destination unreachable messages changes significantly during an attack.

In the last part of our analysis, we chose to calculate the mutual information between the features and the target ('class' column): "The mutual information (MI) between two quantities measures the extent to which knowledge of one quantity reduces uncertainty about the other [...] MI can help you understand the relative potential of a feature as a predictor of the target, considered by itself" [14]. In the next image (figure 2), we show the Mutual Information between the features of the training dataset.

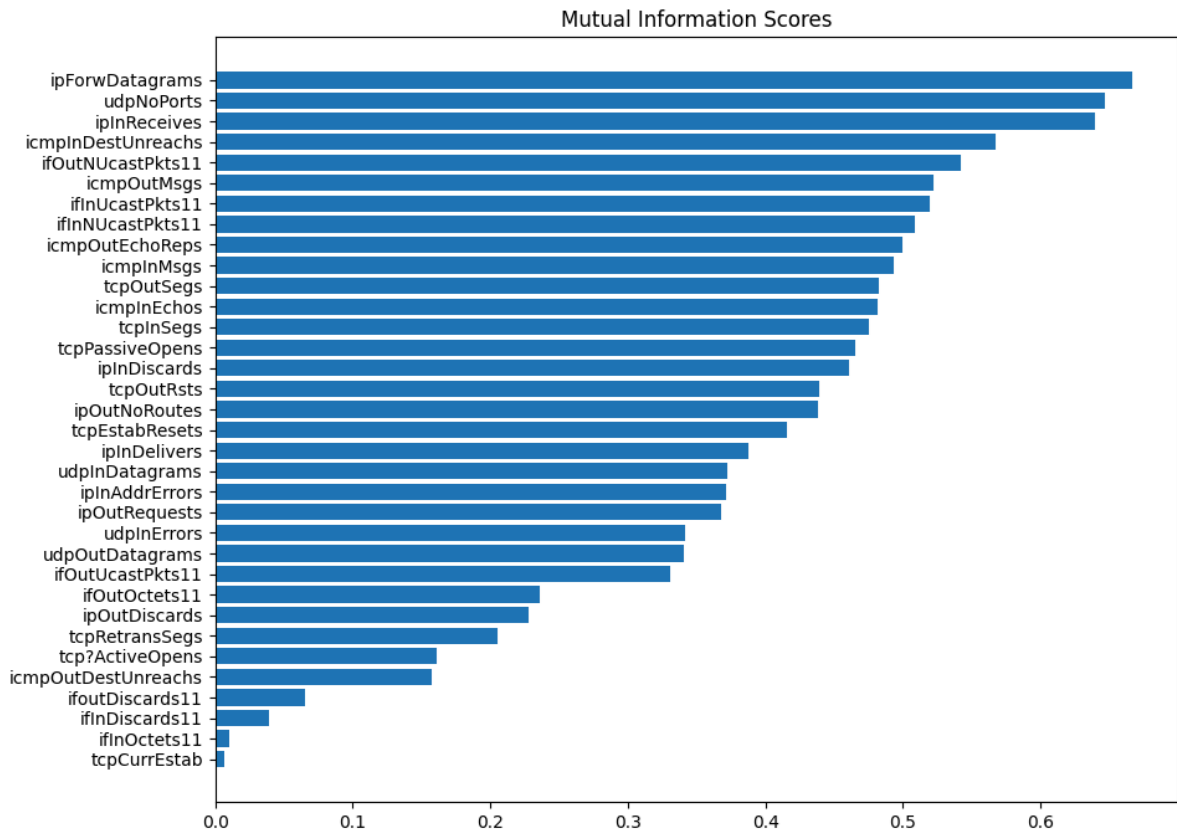


Figure 2. Mutual Information between the features and the target.

From this mutual information analysis, we can observe that among the 10 features with the highest results, two are from the IP group ('ipInReceives' and 'ipForwDatagrams'), one from the UDP group ('udpNoPorts'), three from the Interface group ('ifOutNUcastPkts11', 'ifInNUcastPkts11', and 'ifInNUcastPkts11'), and four from the ICMP group ('icmpInDestUnreachs', 'icmpOutMsgs', 'icmpInMsgs', and 'icmpOutEchoReps'). The 10 features with the lowest mutual information include one from the UDP group ('udplnErrors'), four from the Interface group ('ifOutOctets11', 'ifOutDiscards11', 'ifInDiscards11', and 'ifInOctets11'), three from the TCP group ('tcpRetransSegs', 'tcpActiveOpens', and 'tcpCurrEstab'), one from the IP group ('ipOutDiscards'), and one from the ICMP group ('icmpOutDestUnreachs') (figure 2).

The last part of this section, as we are proposing a closed-loop system with automated networks, is to show an option to use Python to automate this analysis and generate reports. For this case, we used the following tools: Matplotlib and Seaborn (data visualization tools), Jinja2 (to create an HTML template),

Schedule, Time, and OS. In this analysis report, we plot a line graph, the descriptive statistics of the data, the correlation matrix, and the mutual information of the data, this analysis report is generated every day.

4. Troubleshooting

In this section, we are going to use advanced analytics to troubleshoot network problems. After collecting, processing, and analyzing the SNMP data, we will build a machine learning model to classify the data according to the functioning of the network—whether it is normal or under each type of attack. ML models can be part of IDS systems:

ML-based IDS provides a learning-based system that classifies possible attacks based on the behavior of incoming packets. The advantages of ML-based IDS over conventional signature-based IDS are as follows: flexible rather than rule-based; low computational load; ability to detect novel attacks; ability to capture complex properties of the attack behavior; and clustering ML-based IDS does not require updates on the dataset [15].

As our dataset is labeled, we are going to train a supervised classification model to predict which class each SNMP record belongs to. We chose to use the SVM algorithm. “SVM aims to find a hyperplane in an N-dimensional space that distinctly classifies the data points [...] To separate several classes of data points, many hyperplanes can be constructed. The main objective of SVM is to find a plane with the maximum distance between the classes [...] SVM can also fit linear and nonlinear data and uses a kernel technique, which is a mathematical construct that can ‘wrap’ the space where the data are located [15].”

As we aim to classify the five classes of the SNMP data, we chose to perform a multiclass classification. Since the SVM model doesn’t support this mode natively, we used the One-versus-One (Ovo) approach. In this approach, we can train a binary classifier for every pair of classes: 0s with 1s, 0s with 2s, 1s with 2s, and so on. This method is more suited to SVM because these algorithms have poor scalability with the size of the training dataset [16]. In our case, with 5 classes, we trained 10 classifiers. To use this approach, we added the parameter *decision_function_shape='ovo'* in Scikit-learn’s SVC (Support Vector Classifier) model.

We established a baseline with class weighting, standard scaling of the numerical features, and feature selection using mutual information (selecting the

15 best features). The evaluation was conducted using stratified k-fold [17] and classification metrics (figure 3).

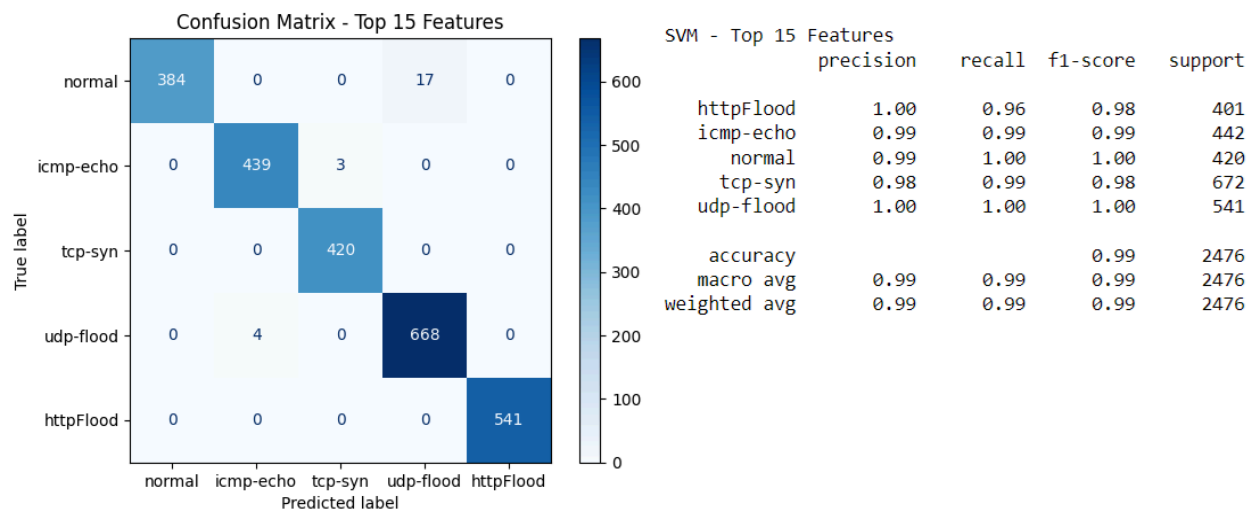


Figure 3. Results of the model's performance

Examining the performance metrics of our baseline, we noticed that in the confusion matrix, the normal class had 17 entries mislabeled as tcp-syn, the largest mislabeled count. Analyzing the metrics by group, we highlight that the 'udp-flood' and 'normal' classes had the best performance, followed by the 'httpFlood' and 'icmp-echo' classes. The 'tcp-syn' class exhibited high precision and recall, with some errors in the identification. The global metrics are also elevated, especially the weighted average, considering the unbalanced classes.

The second part of our troubleshooting section involves using the machine learning model to generate a report with the results of the new data predictions, and some suggestions of actions that can be taken in case of each different attack is detected. Our first step is to create a script with the chosen model, train it, and save it using the joblib library.

In our second script, we deploy the model locally to receive new unlabeled data and generate an alert report with the model's results. We used joblib again to load the model, scaler, and features. Then, we used the Flask framework to build an app for the model. According to its documentation: "Flask enables us to create an app that runs on a web server [...] Entering a URL in a web browser is made operable by a route in a Python decorator"[18].

This way, we used the endpoint for the prediction (which can be accessed via HTTP, and the new data can be uploaded with HTTP POST). We also created a function to generate a report with the classification of the new data and the suggestions for actions (we used Jinja2 again). Lastly, to close our cycle, we scheduled this prediction to run every 5 minutes, using the data collected in our first script, through the timestamps.

In The results of this work, we proposed the closed-loop system with SNMP data and Python. We used several of its libraries and frameworks to build an automated system that collects, processes, analyzes data, and detects attacks, generating reports with actions to be adopted.

DISCUSSION

Our investigation demonstrates the powerful capabilities of Python in network automation and security through the use of SNMP data. By leveraging Python's extensive libraries and frameworks (such as PySNMP, pandas, scikit-learn, and Flask), we proposed a closed-loop automated IDS system. The results show that SNMP data, when processed and analyzed correctly, can provide detailed insights into network behavior, significantly improving monitoring and security measures. The baseline classification model's results were satisfactory, and the data analysis revealed significant correlations and variances in network metrics under different types of attacks.

The comprehensive data collection using the PySNMP library provided a detailed view of network performance across multiple layers of the OSI model. The subsequent data processing with pandas allowed for effective cleaning, normalization, and standardization, ensuring high-quality input for our machine-learning models. Our univariate and bivariate analyses highlighted critical changes in network behavior during attacks, providing valuable indicators for anomaly detection.

We also concluded that SNMP data is a rich resource for diagnosing network issues and, when combined with machine learning techniques, can significantly enhance network monitoring and security. By utilizing the fine-grained data provided by MIBs in anomaly detection, some challenges in network intrusion detection can be mitigated. For every malicious activity, a specific MIB variable

will be affected in some way, making SNMP-MIB data a valuable source of indicators for anomaly detection [11].

CONCLUSION

In this work, we have demonstrated the effectiveness of using Python in combination with SNMP data for network automation and artificial intelligence applications. Our results show that Python's versatility and the extensive range of libraries available make it a powerful tool for implementing network automation, maintenance, and security measures.

As proposed in our objectives, we discussed the main aspects of network automation, including its concepts, use cases, advantages, and challenges. We highlighted various scenarios where automation can significantly improve network performance, going beyond the common understanding of the topic. Furthermore, we proposed a closed-loop system that integrates several use cases, including data collection, data processing, data analysis, and troubleshooting.

Our investigation confirmed that Python, with its rich ecosystem of libraries such as PySNMP, pandas, matplotlib, scikit-learn, and Flask, is highly effective for developing network automation solutions. We demonstrated this through our proposed closed-loop system, which collects data via SNMP, processes and analyzes it, and applies machine-learning techniques to detect network anomalies. This system leverages Python's capabilities to automate and enhance network management tasks, providing real-time insights and responses.

All the objectives outlined in our introduction were addressed. However, as the title of our work suggests, we proposed an IDS automated system using SNMP data in Python, and future steps are necessary to fully implement and evaluate this system in a real network environment. The next steps include connecting this system to an actual network to collect real-time data, running the scripts, and evaluating the model with new data. Additionally, further improvements can be made to the model by performing feature engineering, hyperparameter tuning, testing other classification algorithms, and conducting a more detailed error analysis of the baseline model.

Beyond improving the model, future work can also focus on enhancing the reporting capabilities, security features, data compression, notification systems, and developing interactive dashboards. Additionally, implementing more robust

troubleshooting measures, such as advanced log analysis, real-time anomaly detection, automated incident response, and integration with network forensics tools, can significantly improve the system's resilience. These enhancements will further solidify the system's utility and effectiveness in real-world network management and security applications.

REFERENCES

- [1] Open Networking Foundation, "Software-Defined Networking: The New Norm for Networks", ONF White Paper, Apr. 13, 2012. [Online]. Available: <https://opennetworking.org/wp-content/uploads/2011/09/wp-sdn-newnorm.pdf>
- [2] P. Mihăilă, T. Bălan, R. Curpen, and F. Sandu, "Network Automation and Abstraction using Python Programming Methods", 2017. [Online]. Available: https://www.researchgate.net/publication/322017645_Network_Automation_and_Abstraction_using_Python_Programming_Methods
- [3] K. Dang, "Network Automation With Python", 2023. [Online]. Available: https://www.theseus.fi/bitstream/10024/802362/2/Khoa_Dang.pdf&ved=2ahUKEwjKjdSioxOGAxVN_gIHHTQ7CA8QFnoECB4QAQ&usg=AOvVaw11Whzvq1CPPdc5igNGII0
- [4] J. Edelman, S. Lowe, and M. Oswalt, Network Programmability and Automation. Sebastopol, CA: O'Reilly Media, Inc., 2017.
- [5] Cisco, "What is Network Automation?". [Online]. Available: <https://www.cisco.com/c/en/us/solutions/automation/network-automation.html>
- [6] G. Milios, "Network Automation using Python", 2020. [Online]. Available: <https://repository.ihu.edu.gr/xmlui/handle/11544/29802>
- [7] M. Alkasassbeh, "SNMP 2016 dataset", Mendeley Data, V1, doi: 10.17632/krbhsg5xrt.1. [Online]. Available: <https://data.mendeley.com/datasets/krbhsg5xrt/1>
- [8] S. Prasad, D. Behl, J. Raju, and T. Lewis, "Building AI-driven closed-loop automation systems", 2020. [Online]. Available: <https://developer.ibm.com/articles/an-introduction-to-closed-loop-automation/>
- [9] PySNMP GitHub page. [Online]. Available: <https://github.com/etingof/pysnmp>
- [10] R. F. da Cunha Carvalho, "Plataforma para Monitorização de um Data Center", 2016. [Online]. Available: <https://repositorio.utad.pt/items/bf610e47-c628-40c0-8066-e9ae862bfef0>

- [11] M. Alkasassbeh, G. Al-Naymat, and E. Hawari, "Towards generating realistic SNMP-MIB Dataset for anomaly detection", International Journal of Computer Science and Information Security (IJCSIS), vol. 14, no. 9, 2016. [Online]. Available: https://www.researchgate.net/publication/310518500_Towards_Generating_Realistic_SNMP-MIB_Dataset_for_Network_Anomaly_Detection
- [12] Pandas documentation. [Online]. Available: https://pandas.pydata.org/docs/getting_started/overview.html
- [13] Scikit-learn documentation. [Online]. Available: <https://scikit-learn.org/stable/index.html>
- [14] R. Holbrook, "Mutual Information", Kaggle. [Online]. Available: <https://www.kaggle.com/code/ryanholbrook/mutual-information>
- [15] A. Gerón, Hands-on Machine Learning with Scikit-Learn, Keras and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems, 2nd ed. Sebastopol, CA: O'Reilly, 2019.
- [16] M. A. Ridwan, N. A. M. Radzi, F. Abdullah, and Y. E. Jalil, "Applications of Machine Learning in Networking: A Survey of Current Issues and Future Challenges", IEEE Access, vol. 9, pp. 52523-52556, 2021, doi: 10.1109/ACCESS.2021.3069210. [Online]. Available: <https://ieeexplore.ieee.org/document/9388670>
- [17] J. Olamendy, "A Comprehensive Guide to Stratified K-Fold Cross-Validation for Unbalanced Data", Medium. [Online]. Available: <https://medium.com/@juanc.olamendy/a-comprehensive-guide-to-stratified-k-fold-cross-validation-for-unbalanced-data-014691060f17>
- [18] Flask documentation, "What you know now about Flask", [Online]. Available: <https://python-adv-web-apps.readthedocs.io/en/latest/flask.html#what-you-know-now-about-flask>